Lecture Notes: Application Deployment

This module focuses on how serverless computing, version control and container orchestration enable scalable and efficient deployment of modern data applications. You will learn how to build event-driven workflows, automate ETL pipelines, deploy APIs, and manage applications at scale using cloud-native services and Kubernetes.

By the end of the module, you will be able to:

- Explain serverless architecture and how it differs from traditional systems
- Deploy serverless workflows on GCP
- Use FastAPI for batch and API-driven processing
- Manage code and collaboration using Git
- Deploy containerised applications on Kubernetes

Session: Serverless Deployment

What Serverless Deployment Means

Serverless does not mean "no servers exist"; it means **you do not manage servers**. The cloud provider handles:

- Provisioning
- Scaling
- High availability
- Resource lifecycle management

You deploy only your code. The provider executes it when triggered.

Key Benefits of Serverless

- **Automatic Scalability** – Functions scale based on real-time demand.
- **Cost Efficiency** – You pay only when your code runs.
- **Operational Simplicity** – No configuration of servers or runtime environments.
- **Event Driven** – Functions run in response to triggers like file uploads, API calls, or timers.

Comparing Serverless Across Cloud Providers

AWS Lambda

- Strong integration with AWS ecosystem
- Wide range of triggers
- Mature tooling

Azure Functions

- Strong developer tooling
- Tight integration with Azure services

Google Cloud Functions

- Global availability
- Transparent pricing
- Simple deployment process

Each cloud provider offers similar concepts but differs in ecosystem integration and tooling.

Dependency Management & Environment Handling

Serverless functions often require external libraries, environment variables, and runtime configuration.

Dependency Management

- Package dependencies along with code
- OR use tools like:
    - **Serverless Framework**
    - **AWS SAM**
    - **Terraform** (generic IaC approach)

Environment Handling

- Store secrets and configs in environment variables
- Configure memory, timeout and concurrency settings
- Use encrypted storage (Secret Manager / Key Vault)

Logging in Serverless Environments

Serverless logs are captured by cloud-managed logging systems, not by local machines.

Logging Systems by Provider

- **AWS:** CloudWatch Logs
- **Azure:** Azure Monitor + Application Insights
- **GCP:** Cloud Logging

Logging helps with:

- Debugging
- Performance measurement
- Cost optimisation

Monitoring in Serverless Environments

Monitoring provides a high-level view of serverless application health.

Common Metrics

- Invocation count
- Execution duration
- Error rates
- Cold starts
- Resource consumption

Distributed Tracing

- Tracks request flows across multiple services
- Helps identify performance bottlenecks

Tools used for tracing include:

- AWS X-Ray
- Azure Application Insights
- GCP Operations Suite

Session: Serverless Deployment in GCP

This session demonstrates how to build an end-to-end serverless ETL workflow on GCP using:

- Cloud Functions
- FastAPI
- GCS (Google Cloud Storage)
- Dataproc
- BigQuery

Deploying a FastAPI Serverless Endpoint Triggered by GCS

This workflow allows GCS file uploads to trigger a FastAPI endpoint via Cloud Functions.

Concept

1. A file is uploaded to a GCS bucket.
2. A GCS trigger activates a Cloud Function.
3. The Cloud Function sends an HTTP request to a FastAPI endpoint.
4. FastAPI processes the event (validation, metadata extraction, etc.).

Step-by-Step: Deploying the Workflow

Step 1 — Prepare FastAPI App

```
from fastapi import FastAPI


app = FastAPI()


@app.post("/process-file")
def process(file_name: str):
  return {"status": "received", "file": file_name}
```

Step 2 — Deploy FastAPI on Cloud Run

1. Package the app in a Dockerfile
2. Deploy using:

3. gcloud run deploy fastapi-service --source .

Step 3 — Create a Cloud Function Triggered by GCS

- Choose “Cloud Storage Trigger”
- Event: **Finalize/Create**
- Runtime: Python

Step 4 — In the Cloud Function, call FastAPI

```
import requests


def trigger_fastapi(event, context):
  file_name = event['name']
  requests.post(
    "YOUR_FASTAPI_URL/process-file",
    json={"file_name": file_name}
  )
```

Triggering PySpark ETL Jobs on Dataproc

After FastAPI processes metadata, another Cloud Function triggers a Dataproc ETL job.

Step-by-Step Workflow

Step 1 — Upload PySpark script to GCS

Example file: gs://my-bucket/etl/transform.py

Step 2 — Configure a Dataproc Cluster or Use Serverless Mode

Step 3 — Cloud Function submits Dataproc job

```
from google.cloud import dataproc_v1


def start_etl(event, context):
  job_client = dataproc_v1.JobControllerClient()
  job = {
    "pyspark_job": {
```

```
        "main_python_file_uri": "gs://my-bucket/etl/transform.py"
    }
}
job_client.submit_job(
    request={"project_id": "YOUR_ID", "region": "us-central1", "job": job}
)
```

Writing Output to BigQuery or GCS

After Dataproc processes data, output can be:

Option 1 — Written to BigQuery

- Load into an analytics-ready table
- Supports SQL processing

Option 2 — Written to GCS

- Common formats: **CSV**, **Parquet**
- Good for archiving or raw zones

Session: Batch Processing with FastAPI API

This workflow uses FastAPI + Cloud Functions to process ETL requests in batch mode.

Creating an HTTP-Triggered Cloud Function for Batch ETL

Concept

1. A user sends an HTTP request to a Cloud Function.
2. The function validates inputs.
3. It triggers a PySpark job on Dataproc.
4. It returns a response once the job is submitted.

Step-by-Step Workflow

Step 1 — Create a FastAPI endpoint

@app.post("/batch-etl")

```
def batch_etl(input_file: str):
    return {"message": "ETL started", "file": input_file}
```

Step 2 — Cloud Function receives HTTP request

```
def http_trigger(request):
    payload = request.get_json()
    file_path = payload["file_path"]
```

Step 3 — Cloud Function starts Dataproc ETL

(Same as earlier Dataproc submission step)

Step 4 — Return job submission confirmation

```
return {"status": "submitted"}
```

Session: Version Control with Git

Git enables versioning of code, notebooks and configuration files for collaboration and reproducibility.

Git Essentials for Data Workflows

Key Commands

- git init — start versioning a project
- git add . — stage changes
- git commit -m "message" — snapshot changes
- git branch <name> — create isolated environments
- git merge <branch> — integrate changes

Git LFS for Large Data Files

Why Git LFS is needed

Large data files slow down repositories and cause bloated history.

Steps to Use Git LFS

1. Install LFS:
2. git lfs install
3. Track files:
4. git lfs track "*.csv"
5. Commit normally:
6. git add data.csv
7. git commit -m "added large file"

Git stores lightweight pointers while hosting actual files remotely.

Structuring Data Projects

Recommended folders:

- **data/** – datasets (use LFS for large files)
- **notebooks/** – exploratory notebooks
- **scripts/** – Python scripts, ETL pipelines
- **models/** – trained model artifacts
- **configs/** – YAML/env files

A structured layout ensures maintainability and clarity.

Version Control of Notebooks

Challenges arise due to metadata and outputs.

Best Practices

- Clear notebook outputs before committing
- Use nbdime for meaningful diffs
- Avoid committing large cell outputs

Branching Strategies for Collaboration

Git Flow

Separate branches for:

- production
- development
- feature branches

Trunk-Based Development

- Frequent commits to main branch
- Continuous integration
- Fast collaboration cycles

Session: Container Orchestration with Kubernetes

Kubernetes Fundamentals

Kubernetes automates deployment and scaling of containerised applications.

Core Components

- **Pod** — smallest deployable unit
- **Deployment** — manages replica count and rollout strategy
- **Service** — provides network access to pods
- **ConfigMap** — store non-sensitive configuration
- **Secret** — store sensitive configuration
- **Volume** — persistent storage
- **Helm Chart** — application packaging system

These components work together to ensure reliability, scalability and resilience.

Kubernetes on Cloud (With EKS & AKS Removed)

Managed Kubernetes shifts the control plane to the cloud provider.

Focus: GKE (Google Kubernetes Engine)

- Automated node provisioning
- Integration with GCP services (Cloud Storage, BigQuery)
- Built-in monitoring and logs

Deploying FastAPI on Kubernetes

Key Deployment Concepts

- Docker image stores the FastAPI app
- Deployment defines replicas, container image and restart policy
- Service exposes the deployment to internal/external traffic
- Ingress enables domain-based routing

FastAPI Deployment on GKE

Step-by-Step Workflow

Step 1 — Containerise the FastAPI App

Dockerfile:

```
FROM python:3.10-slim

WORKDIR /app

COPY . .

RUN pip install fastapi uvicorn

CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "80"]
```

Step 2 — Build & Push Image

```
gcloud builds submit --tag gcr.io/PROJECT_ID/fastapi
```

Step 3 — Create Kubernetes Deployment

```
apiVersion: apps/v1

kind: Deployment

metadata:

 name: fastapi-deployment

spec:

 replicas: 3

 selector:

  matchLabels:

   app: fastapi

 template:
```

```
  metadata:
   labels:
    app: fastapi
  spec:
   containers:
   - name: fastapi
    image: gcr.io/PROJECT_ID/fastapi
    ports:
    - containerPort: 80
```

Step 4 — Expose via Service

```
kind: Service
apiVersion: v1
metadata:
 name: fastapi-service
spec:
 type: LoadBalancer
 selector:
  app: fastapi
 ports:
 - port: 80
  targetPort: 80
```

Step 5 — Access the Application

- Retrieve external IP from LoadBalancer
- Test API through browser or Postman

Ask an LLM

1. What advantages does serverless architecture offer over container-based deployments?

2. How do Cloud Functions coordinate multi-step pipelines in GCP?
3. Compare logging and monitoring strategies in serverless vs containerised environments.
4. How does Git LFS improve repository performance in data science workflows?
5. Why is Kubernetes preferred for large-scale API deployments?
6. Explain how FastAPI integrates into both serverless and Kubernetes architectures.

Topics Commonly Asked in Interviews

- Difference between serverless, containers and VMs
- Cloud Functions vs Cloud Run
- When to choose Dataproc serverless vs cluster mode
- Common CI/CD steps for deploying FastAPI applications
- Git branching strategies in collaborative projects
- Kubernetes deployment lifecycle
- Autoscaling strategies in Kubernetes
- Designing event-driven ETL pipelines on GCP
- Handling large data files in Git using LFS
- Best practices for structuring data projects